PRAKTIKUM BASIS DATA

MODUL 2

DDL DML

Oleh:

Najwa Areefa Ghaisani 103122400028

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

1. **Buatlah setiap tabel diatas, tuliskan script SQL nya dan sertakan tangkapan layarnya**

   Jawaban:

Worksheet | Query Builder

```sql
CREATE TABLE FILM (
    id_film NUMBER PRIMARY KEY,
    judul VARCHAR2(100),
    sinopsis VARCHAR2(250),
    tahun NUMBER,
    durasi NUMBER
);

CREATE TABLE THEATER (
    id_theater NUMBER PRIMARY KEY,
    harga NUMBER,
    kapasitas NUMBER,
    kelas VARCHAR2(50)
);

CREATE TABLE MEMBER (
    id_member INT PRIMARY KEY,
    nama VARCHAR2(100),
    no_hp VARCHAR2(15),
    tgl_lahir DATE,
    email VARCHAR2(100)
);
```

Worksheet | Query Builder

```sql
CREATE TABLE JADWAL (
    id_jadwal NUMBER PRIMARY KEY,
    id_film NUMBER,
    id_theater NUMBER,
    periode_start DATE,
    periode_end DATE,
    FOREIGN KEY (id_film) REFERENCES FILM(id_film),
    FOREIGN KEY (id_theater) REFERENCES THEATER(id_theater)
);

CREATE TABLE TRANSAKSI (
    kode_pemesanan NUMBER PRIMARY KEY,
    id_jadwal NUMBER,
    id_member NUMBER,
    status VARCHAR2(20),
    tanggal DATE,
    total_harga NUMBER,
    FOREIGN KEY (id_jadwal) REFERENCES JADWAL(id_jadwal),
    FOREIGN KEY (id_member) REFERENCES MEMBER(id_member)
);

CREATE TABLE INVENTARIS (
    id_inventaris NUMBER PRIMARY KEY,
    id_theater NUMBER,
    nomor_kursi NUMBER,
    FOREIGN KEY (id_theater) REFERENCES THEATER(id_theater)
);
```

Penjelasan:

Jadi di sini aku bikin 6 tabel yang dibutuhin buat sistem bioskop. Pertama tabel FILM buat nyimpen data film kayak judul, sinopsis, tahun sama durasi. Terus tabel THEATER buat data studio/ruangan bioskopnya, ada harga tiket, kapasitas sama kelasnya (Regular, VIP, VVIP). Tabel MEMBER buat data penonton yang udah daftar sebagai member. Tabel JADWAL buat jadwal tayang filmnya di theater mana dan tanggal berapa, di sini juga udah aku tambahin FOREIGN KEY ke tabel FILM dan THEATER supaya datanya nyambung. Tabel TRANSAKSI buat nyimpen data

pemesanan tiket, juga ada FOREIGN KEY ke JADWAL sama MEMBER. Terakhir tabel INVENTARIS buat data kursi di tiap theater, relasinya ke tabel THEATER. Intinya semua tabel udah dihubungin lewat foreign key biar datanya konsisten dan nggak ada data yang 'nyasar'.

2. **Lakukan insert data pada ke-enam tabel, sertakan tangkapan layar dan keterangannya**
   Jawaban:

   1. **Insert Table Film**

Script:

```
INSERT INTO FILM VALUES (1, 'The Call', 'Film Horror Thriller Korea', 2020, 180);
INSERT INTO FILM VALUES (2, 'The Witch: Part 1', 'Film Action Mystery', 2018, 180);
INSERT INTO FILM VALUES (3, 'Parasite', 'Film Drama Thriller Korea', 2019, 132);
```

Output:

Script Output × | Query Result ×

SQL | All Rows Fetched: 3 in 0.009 seconds

	ID_FILM	JUDUL	SINOPSIS	TAHUN	DURASI
1	1	The Call	Film Horror Thriller Korea	2020	180
2	2	The Witch: Part 1	Film Action Mystery	2018	180
3	3	Parasite	Film Drama Thriller Korea	2019	132

Penjelasan: Di sini aku masukin 3 data film pake perintah INSERT INTO FILM VALUES. Film pertama 'The Call' (2020) bergenre Horror Thriller Korea dengan durasi 180 menit, kedua 'The Witch: Part 1' (2018) genre Action Mystery juga 180 menit, dan ketiga 'Parasite' (2019) genre Drama Thriller Korea berdurasi 132 menit. Nilai yang dimasukin urutannya harus sesuai sama kolom yang ada di tabel, jadi id_film dulu, terus judul, sinopsis, tahun, baru durasi. Dari outputnya keliatan 3 baris data berhasil masuk ke tabel FILM.

2. Insert Table Theater

Script:

```
INSERT INTO THEATER VALUES (1, 50000, 120, 'Regular');
INSERT INTO THEATER VALUES (2, 100000, 80, 'VIP');
INSERT INTO THEATER VALUES (3, 150000, 50, 'VVIP');
```

Output:

SQL | All Rows Fetched: 3 in 0.007 seconds

	ID_THEATER	HARGA	KAPASITAS	KELAS
1	1	50000	120	Regular
2	2	100000	80	VIP
3	3	150000	50	VVIP

Penjelasan: Tabel THEATER diisi dengan 3 jenis kelas studio. Theater 1 adalah kelas Regular dengan harga tiket 50.000 dan kapasitas 120 kursi, theater 2 kelas VIP harga 100.000 kapasitas 80 kursi, dan theater 3 kelas VVIP harga 150.000 kapasitas 50 kursi. Harga tiket yang berbeda-beda ini disesuaiin sama kelasnya, makin premium kelasnya ya makin mahal tiketnya tapi kapasitas kursinya juga makin sedikit. Data berhasil masuk semua, terbukti dari outputnya yang nampillin 3 baris data.

3. Insert Table Member

Script:

```
INSERT INTO MEMBER VALUES (1, 'Najwa', '08123456789',
    TO_DATE('2006-01-01','YYYY-MM-DD'), 'najwa@email.com');
INSERT INTO MEMBER VALUES (2, 'Nadia', '08987654321',
    TO_DATE('2004-05-15','YYYY-MM-DD'), 'nadia@email.com');
INSERT INTO MEMBER VALUES (3, 'Sukma', '08111222333',
    TO_DATE('2005-08-20','YYYY-MM-DD'), 'sukma@email.com');
```

Output:

Script Output | Query Result

SQL | All Rows Fetched: 3 in 0.007 seconds

	ID_MEMBER	NAMA	NO_HP	TGL_LAHIR	EMAIL
1	1	Najwa	08123456789	01-01-2006	najwa@email.com
2	2	Nadia	08987654321	15-05-2004	nadia@email.com
3	3	Sukma	08111222333	20-08-2005	sukma@email.com

Penjelasan: Di tabel MEMBER aku masukin 3 data anggota. Member pertama namanya Najwa dengan nomor HP 08123456789, tanggal lahir 01-01-2006, dan email najwa@email.com. Member kedua Nadia, HP 08987654321, lahir 15-05-2004, email nadia@email.com. Member ketiga Sukma, HP 08111222333, lahir 20-08-2005, email sukma@email.com. Kolom id_member bertipe INT dan jadi PRIMARY KEY, jadi tiap member punya ID unik yang nggak boleh sama. Dari output bisa dilihat ketiga data member berhasil tersimpan di database.

4. Insert Table Jadwal

Script:

```
INSERT INTO JADWAL VALUES (1, 1, 1,
    TO_DATE('2026-03-10','YYYY-MM-DD'),
    TO_DATE('2026-03-20','YYYY-MM-DD'));
INSERT INTO JADWAL VALUES (2, 2, 2,
    TO_DATE('2026-03-11','YYYY-MM-DD'),
    TO_DATE('2026-03-21','YYYY-MM-DD'));
INSERT INTO JADWAL VALUES (3, 3, 3,
    TO_DATE('2026-03-12','YYYY-MM-DD'),
    TO_DATE('2026-03-22','YYYY-MM-DD'));
```

Output:

Script Output | Query Result

SQL | All Rows Fetched: 3 in 0.004 seconds

	ID_JADWAL	ID_FILM	ID_THEATER	PERIODE...	PERIODE...
1	1	1	1	10-03-2026	20-03-2026
2	2	2	2	11-03-2026	21-03-2026
3	3	3	3	12-03-2026	22-03-2026

Penjelasan: Tabel JADWAL berisi informasi kapan dan di mana suatu film ditayangkan. Jadwal 1 adalah film id 1 (The Call) diputar di theater id 1 (Regular) mulai 10 Maret 2026 sampai 20 Maret 2026. Jadwal 2 film id 2 (The Witch) di theater id 2 (VIP) tanggal 11-21 Maret 2026. Jadwal 3 film id 3 (Parasite) di theater id 3 (VVIP) tanggal 12-22 Maret 2026. Tabel ini nggak bisa diisi sembarangan karena ada foreign key, artinya id_film yang dimasukin harus udah ada di tabel FILM, dan id_theater harus udah ada di tabel THEATER. Makanya tabel FILM dan THEATER harus diisi dulu sebelum insert ke sini.

5. Insert Table Transaksi

Script:

```
INSERT INTO TRANSAKSI VALUES (1, 1, 1, 'Lunas',
    TO_DATE('2026-03-10','YYYY-MM-DD'), 50000);
INSERT INTO TRANSAKSI VALUES (2, 2, 2, 'Lunas',
    TO_DATE('2026-03-11','YYYY-MM-DD'), 100000);
INSERT INTO TRANSAKSI VALUES (3, 3, 3, 'Lunas',
    TO_DATE('2026-03-12','YYYY-MM-DD'), 150000);
```

Output:

Query Result

SQL | All Rows Fetched: 3 in 0.191 seconds

	KODE_PEMESANAN	ID_JADWAL	ID_MEMBER	STATUS	TANGGAL	TOTAL_HARGA
1	1	1	1	Lunas	10-03-2026	50000
2	2	2	2	Lunas	11-03-2026	200000
3	3	3	3	Lunas	12-03-2026	150000

Penjelasan: Tabel TRANSAKSI nyimpen data pemesanan tiket oleh member. Transaksi pertama (kode 1) adalah pemesanan jadwal 1 oleh member 1 (Najwa),

statusnya Lunas, tanggal 10 Maret 2026 dengan total harga 50.000. Transaksi kedua (kode 2) oleh member 2 (Nadia) untuk jadwal 2, Lunas, tanggal 11 Maret 2026, total harga 200.000. Transaksi ketiga (kode 3) oleh member 3 (Sukma) untuk jadwal 3, Lunas, 12 Maret 2026, total harga 150.000. Tabel ini juga punya foreign key ke JADWAL dan MEMBER, jadi pastiin dua tabel itu udah ada datanya sebelum insert di sini. Output nunjukkin 3 baris transaksi berhasil masuk.

6. Insert Table Inventaris

Script:

```
INSERT INTO INVENTARIS VALUES (1, 1, 101);
INSERT INTO INVENTARIS VALUES (2, 1, 102);
INSERT INTO INVENTARIS VALUES (3, 2, 201);
```

Output:

Script Output × | Query Result ×

SQL | All Rows Fetched: 3 in 0.005 seconds

	ID_INVENTARIS	ID_THEATER	NOMOR_KURSI
1	1	1	101
2	2	1	102
3	3	2	201

Penjelasan: Tabel INVENTARIS dipakai buat nyimpen data nomor kursi yang ada di tiap theater. Inventaris 1: kursi nomor 101 di theater 1. Inventaris 2: kursi nomor 102 juga di theater 1. Inventaris 3: kursi nomor 201 di theater 2. Jadi satu theater bisa punya banyak data kursi di tabel ini. Ada foreign key ke tabel THEATER, jadi id_theater yang dimasukin harus beneran ada di tabel THEATER. Dari output kelihatan 3 data kursi berhasil tersimpan.

3. **Lakukan update data (minimal 5 data), tuliskan script SQL nya dan sertakan tangkapan layarnya**

1. **Update pada table film**

Script:

```
UPDATE FILM SET judul = 'The Call (2020)' WHERE id_film = 1;
```

Output:

	ID_FILM	JUDUL	SINOPSIS	TAHUN	DURASI
1	1	The Call (2020)	Film Horror Thriller Korea	2020	180
2	2	The Witch: Part 1	Film Action Mystery	2018	180
3	3	Parasite	Film Drama Thriller Korea	2019	132

Penjelasan: Di sini aku update judul film dengan id_film = 1. Awalnya judulnya 'The Call', aku ubah jadi 'The Call (2020)' supaya lebih lengkap dan keliatan tahun rilisnya. Perintah UPDATE yang dipakai: UPDATE FILM SET judul = 'The Call (2020)' WHERE id_film = 1. Kata WHERE itu fungsinya buat nentuin baris mana yang mau diubah. Kalau nggak pakai WHERE, semua judul film di tabel bisa keubah semua, yang jelas nggak kita mau. Dari output bisa dilihat baris pertama judulnya udah berubah jadi 'The Call (2020)' sementara data lainnya tetap sama.

2. **Update pada table Theater**

Script:

```
UPDATE THEATER SET harga = 75000 WHERE id_theater = 1;
```

Output:

	ID_THEATER	HARGA	KAPASITAS	KELAS
1	1	75000	120	Regular
2	2	100000	80	VIP
3	3	150000	50	VVIP

Penjelasan: Di sini aku update harga tiket untuk theater dengan id_theater = 1 (kelas Regular). Harga awalnya 50.000 diubah jadi 75.000. Scriptnya: UPDATE

THEATER SET harga = 75000 WHERE id_theater = 1. Mungkin alasannya kayak ada kenaikan harga tiket bioskop hehe. Dari output tabelnya, harga theater 1 sekarang udah jadi 75.000, sedangkan theater 2 dan 3 tetap 100.000 dan 150.000. Ini membuktikan perintah UPDATE dengan WHERE bekerja dengan benar, cuma ngubah baris yang sesuai kondisi aja.

3. **Update pada table Member**

Script:

```
UPDATE MEMBER SET no_hp = '08199999999' WHERE id_member = 1;
```

Output:

	ID_MEMBER	NAMA	NO_HP	TGL_LAHIR	EMAIL
1	1	Najwa	08199999999	01-01-2006	najwa@email.com
2	2	Nadia	08987654321	15-05-2004	nadia@email.com
3	3	Sukma	08111222333	20-08-2005	sukma@email.com

Penjelasan: Di sini aku update nomor HP member pertama (Najwa) yang awalnya 08123456789 menjadi 08199999999. Scriptnya: UPDATE MEMBER SET no_hp = '08199999999' WHERE id_member = 1. Ini simulasi kondisi di mana member ganti nomor HP, jadi datanya perlu diperbarui. Dari output kelihatan no_hp Najwa (id_member 1) sudah berubah jadi 08199999999, sementara data Nadia dan Sukma tetap seperti semula. Kolom lain kayak nama, tgl_lahir, dan email juga nggak ikut berubah.

4. **Update pada table Transaksi**

Script:

```
UPDATE TRANSAKSI SET status = 'Lunas' WHERE kode_pemesanan = 3;
UPDATE TRANSAKSI SET total_harga = 200000 WHERE kode_pemesanan = 2;
```

Output:

	KODE_PEMESANAN	ID_JADWAL	ID_MEMBER	STATUS	TANGGAL	TOTAL_HARGA
1	1	1	1	Lunas	10-03-2026	50000
2	2	2	2	Lunas	11-03-2026	200000

Penjelasan: Di soal ini ada dua perintah UPDATE sekaligus untuk tabel TRANSAKSI. Pertama, aku update status transaksi dengan kode_pemesanan = 3 menjadi 'Lunas' (misalkan sebelumnya belum lunas). Kedua, aku update total_harga transaksi kode 2 dari yang sebelumnya menjadi 200.000. Dari output keliatan transaksi kode 1 dan 2 masih ada dengan status Lunas, total harga transaksi 2 juga udah jadi 200.000. Sementara transaksi kode 3 nggak muncul di output ini karena pada soal nomor 4 nanti akan dihapus. Jadi ini wajar, update berhasil dijalankan sebelum data dihapus.

4. **Lakukan hapus data (minimal 2 data), tuliskan script SQL nya dan sertakan tangkapan layarnya**

   1. **Delete transaksi dengan kode pesanan 3**

Script:

```
DELETE FROM TRANSAKSI WHERE kode_pemesanan = 3;
```

Output:

	KODE_PEMESANAN	ID_JADWAL	ID_MEMBER	STATUS	TANGGAL	TOTAL_HARGA
1	1	1	1	Lunas	10-03-2026	50000
2	2	2	2	Lunas	11-03-2026	200000

Penjelasan: Di sini aku hapus data transaksi yang punya kode_pemesanan = 3 menggunakan perintah DELETE FROM TRANSAKSI WHERE kode_pemesanan = 3. Sama kayak UPDATE, perintah DELETE juga harus pakai WHERE supaya yang kehapus cuma baris yang dimaksud, bukan semua data. Setelah dijalankan, dari output tabel TRANSAKSI sekarang cuma tinggal 2 baris: kode 1 dan kode 2. Data kode 3 udah beneran hilang dari tabel. Ini berguna misalnya kalau ada pembatalan pemesanan tiket.

   2. **Delete member dengan id_member 3**

Script:

```
DELETE FROM MEMBER WHERE id_member = 3
```

Output:

	ID_MEMBER	NAMA	NO_HP	TGL_LAHIR	EMAIL
1	1	Najwa	08199999999	01-01-2006	najwa@email.com
2	2	Nadia	08987654321	15-05-2004	nadia@email.com

Penjelasan: Selanjutnya aku hapus data member dengan id_member = 3 yaitu Sukma, menggunakan perintah DELETE FROM MEMBER WHERE id_member = 3. Perlu diperhatiin, sebelum menghapus member ini, transaksi yang terkait dengan id_member 3 juga harus dihapus dulu. Karena kalau nggak, bakal error foreign key constraint karena tabel TRANSAKSI masih ngereferensi id_member tersebut. Makanya urutan delete itu penting, hapus dulu data di tabel yang jadi 'anak' (punya foreign key), baru hapus di tabel 'induk'. Dari output, tabel MEMBER sekarang tinggal Najwa dan Nadia, data Sukma sudah terhapus.

3. **Delete inventaris dengan id_inventaris 3**

Script:

```
DELETE FROM INVENTARIS WHERE id_inventaris = 3;
```

Output:

	ID_INVENTARIS	ID_THEATER	NOMOR_KURSI
1	1	1	101
2	2	1	102

Penjelasan: Yang terakhir aku hapus data inventaris kursi dengan id_inventaris = 3 menggunakan perintah DELETE FROM INVENTARIS WHERE id_inventaris = 3. Data yang dihapus adalah kursi nomor 201 di theater 2. Misalnya alasannya karena kursi itu rusak atau memang sudah nggak dipakai lagi. Dari output, tabel INVENTARIS sekarang cuma punya 2 baris yaitu id 1 (kursi 101 di theater 1) dan id 2 (kursi 102 di theater 1). Proses delete ini lebih mudah karena tabel INVENTARIS nggak jadi referensi foreign key tabel lain, jadi bisa langsung dihapus tanpa perlu khawatir ada error constraint.